

Utilisation d'un simulateur de SoC : renode

■■■ Objectifs

- ☐ se familiariser avec gdb et l'exécution de code sur processeur ;
- ☐ disposez d'un environnement simulé pour l'étude de firmware ;

■■■ Débogage niveau processeur

Soit le programme suivant :

```
#include <stdio.h>

int main()
{
    printf("Hello world\n");
}
```

1 – Question

- a. Trouvez les adresses de l'appel système lié à « write » et dumppez le contenu des registres pour retrouver la chaîne "Hello world".

■■■ Installation de renode

2 – Étudiez la proposition de Renode <https://renode.io/>

- a. comment peut-on suivre l'exécution d'un programme sous Linux ?
- b. quelles différences avec l'embarqué ?
- c. Qu'est-ce que la « cross-compilation » ?
- d. Comment procéder quand il n'y a pas d'OS ?
- e. Quel est l'intérêt de simuler un processeur ?
- f. Quelle est l'originalité de Renode ?
- g. Comparez Qemu et Renode ?

Vous trouverez à https://git.p-fb.net/PeFClic/docker_rp2040 un dockerfile pour installer :

- ▷ le simulateur Renode ;
- ▷ un émulateur de la carte de développement Raspberry Pico avec ses principaux périphériques.

■■■ Simulation de firmware bare-metal

3 – Installation du Pico-sdk :

```
xterm
$ git clone https://github.com/raspberrypi/pico-sdk.git
$ cd pico-sdk
$ git submodule update --init
$ export PICO_SDK_PATH=~/.pico-sdk
```

Installation de PicoTool :

```
xterm
$ git clone https://github.com/raspberrypi/picotool.git
$ cmake -B build .
$ cd build
$ make
```

Installation de firmware bare-metal :

```
xterm
$ git clone https://github.com/carlosftm/RPi-Pico-Baremetal.git
$ cd RPi-Pico-Baremetal
$ ls
$ cd 00_asm_blink
$ make
```

Exécutez et étudiez les firmwares proposés sous renode combiné avec gdb.